

# Textul prezentării disertației

## Slide 2 – Cuprins

În această prezentare voi aborda:

- În introducere, contextul și motivația lucrării
- Tehnologiile utilizate
- Arhitectura generală a sistemului
- Algoritmul Schnorr integrat în protocol
- Implementarea fluxurilor de înregistrare și autentificare
- Proprietățile de securitate
- Testarea funcțională și simularea atacurilor
- Evaluarea performanței, comparativ cu OAuth 2.0 și protocoalele PAKE: SRP-6a și zk-SNARK
- La final, concluziile, limitările și direcțiile de dezvoltare, urmate de bibliografie

## Slide 3 – Introducere

Tema pornește de la o problemă structurală a aplicațiilor web moderne: autentificarea se bazează pe transmiterea unui secret partajat către server. Deși HTTPS reduce riscul interceptării, arhitectura clasică rămâne vulnerabilă în fața compromiterii bazelor de date, a reutilizării parolelor și a atacurilor de tip replay.

Standardul OAuth 2.0, adoptat pe scară largă, nu soluționează în mod intrinsec această problemă, faza de autentificare continuând să expună credențialele în tranzit.

Scopul este proiectarea unui protocol hibrid care îmbină schema de identificare Schnorr, fundamentată pe demonstrații cu cunoștințe zero, cu cadrul OAuth 2.0, astfel încât parola să nu părăsească niciodată dispozitivul clientului. Au fost implementate și trei fluxuri OAuth clasice, inclusiv o variantă PKCE și una bazată pe Authlib, ca bază de referință comparativă, completate de o evaluare față de protocoalele PAKE consacrate, SRP-6a și zk-SNARK.

## Slide 4 – Tehnologii

Python cu Flask a fost ales pentru serverul de autentificare datorită ecosistemului extins de biblioteci criptografice. Flask-SQLAlchemy asigură persistența într-o bază SQLite locală cu trei tabele: utilizatorii cu cheile publice, jetoanele emise și datele protejate. Am utilizat cryptography și sympy pentru parametrii criptografici, PyJWT pentru jetoanele de acces în format JWT semnate HS256, iar Authlib pentru implementarea de referință OAuth 2.0 PKCE.

Suita de testare se bazează pe pytest pentru cazurile funcționale și vectorii de atac, iar Locust pentru evaluarea performanței sub încărcare concurentă.

JavaScript a fost ales pentru client, unde toate calculele criptografice se execută local, prin BigInt și Web Crypto, fără cadre de lucru externe.

## Slide 5 – Arhitectură

Arhitectura este de tip trei niveluri. Nivelul de prezentare cuprinde interfața web statică unde se execută calculele criptografice ZKP pe dispozitivul utilizatorului.

Nivelul de aplicație, conturat de Flask, gestionează logica criptografică, emiterea jetoanelor JWT și modulele comparative OAuth 2.0.

Nivelul de date conține persistența într-o bază SQLite cu cele trei tabele menționate: utilizatorii cu cheile publice (secret\_y), jetoanele emise (AuthToken) și entitățile de date protejate (PersoData). La nivelul bazei de date nu se stochează niciun secret privat.

## Slide 6 – Algoritmul Schnorr

Schema Schnorr este fundamentată pe dificultatea problemei logaritmului discret în grupuri finite. Se utilizează un număr prim sigur  $P$ , unde  $P = 2Q + 1$ , un generator  $G$  al subgrupului de ordin  $Q$ , iar cheia privată  $x$  aparține spațiului  $ZQ$ . Protocolul se desfășoară între un solicitant, care deține secretul, și un verficator, care validează dovada fără a obține informații despre secret. Interacțiunea presupune patru etape: generarea cheii publice  $y = G^x \bmod P$ , transmiterea angajamentului  $t = G^r \bmod P$ , primirea provocării aleatorii  $c$ , calculul răspunsului  $s = (r + c * x) \bmod Q$  și, în final, verificarea egalității  $G^s \bmod P = t * y^c \bmod P$ .

## Slide 7 – Fluxul de înregistrare

În procesul de înregistrare, clientul derivă local cheia privată din parolă utilizând funcția scrypt și calculează cheia publică  $y = G^x \bmod P$ . Către server se transmite exclusiv cheia publică, care este stocată și asociată identității utilizatorului.

Parola și cheia privată nu părăsesc niciodată dispozitivul clientului. În eventualitatea compromiterii bazei de date, valorile publice sunt inutile criptografic, derivarea cheii private fiind echivalentă cu rezolvarea problemei logaritmului discret.

## Slide 8 – Fluxul de autentificare

Clientul generează un nonce aleatoriu  $r$ , calculează angajamentul  $t$  și îl transmite împreună cu identitatea utilizatorului. Serverul validează apartenența la subgrup, creează o sesiune temporară cu un timp de viață de 5 secunde și returnează provocarea aleatorie  $c$ .

Clientul recalculează cheia privată din parolă, calculează răspunsul  $s = (r + c * x) \bmod Q$  și îl transmite. Serverul verifică egalitatea matematică  $G^s \bmod P = t * y^c \bmod P$ , utilizând exclusiv cheia publică stocată. La succes, emite un jeton JWT ancorat de contextul sesiunii ZKP prin mecanismul de legare de canal.

## Slide 9 – Proprietăți de securitate

Rezistența la interceptare: un adversar vizualizează exclusiv transcrisul efemer. Ecuația  $s = (r + c * x) \bmod Q$  nu poate fi rezolvată fără nonce-ul  $r$ , care nu este transmis prin rețea. Protocolul oferă reziliență la atacurile de tip Store Now, Decrypt Later.

Eliminarea secretului partajat: serverul stochează doar cheia publică, neutralizând impactul breșelor de date. Provocarea unică per sesiune previne atacurile replay. Legarea de canal și expirarea JWT atenuează deturnarea sesiunii.

Poza cu Wireshark

## Slide 10–11 – Validare și teste de securitate

Validarea a fost realizată prin suite pytest acoperind 49 de cazuri de test distribuite pe patru categorii: 5 pozitive, 5 negative, 31 cazuri limită și 8 teste de securitate.

Testele pozitive verifică: absența parolei în orice format în baza de date, fluxurile complete de înregistrare și autentificare, autentificarea simultană a mai multor utilizatori.

Testele negative acoperă: tentativa de autentificare cu parolă greșită având doar cheia publică  $y$  fără cheia privată  $x$ , atacul de tip replay, expirarea sesiunii ca prevenire DoS, suprascrierea sesiunii existente și conflictul la înregistrare duplicată.

Cazurile limită testează: respingerea valorilor  $s$  în afara spațiului  $ZQ$ , respingerea valorilor  $y$  și  $t$  din subgrupuri triviale  $(0, 1, -1, P-1)$ , încercări concurente de autentificare, valori neconforme ca tip de dată pentru  $y$ ,  $t$  și  $s$ , și lipsa header-ului de sesiune.

Testele de securitate validează: simulatorul Schnorr, autentificarea cu  $t = y$ , falsificarea  $s$  fără cheia privată, erori off-by-one pe soluție, izolarea sesiunilor, unicitatea valorilor generate și autentificarea cu cheia publică furată.

Auditul de trafic prin Wireshark a confirmat că niciun mesaj HTTP nu conține parola brută sau derivate directe.

## Slide 12 – Evaluarea performanței

Evaluările pe 100 de iterații au indicat o latență medie de verificare criptografică de 33,87 milisecunde pe server, o latență totală a fluxului ZKP complet de 64,76 milisecunde și un debit susținut de aproximativ 15,36 cereri pe secundă. Prin comparație, fluxurile OAuth 2.0 ating între 460 și 1034 cereri pe secundă.

Compromisul temporal este justificat de garanțiile criptografice superioare obținute, iar latența totală rămâne imperceptibilă pentru utilizatorul final în contextul autentificării interactive.

## Slide 13 – Comparație ZKP față de OAuth 2.0

În protocolul ZKP Schnorr, secretul nu traversează niciodată rețeaua. În fluxurile OAuth PKCE, simplificat și Authlib, parola este transmisă în faza de autentificare, chiar dacă restul fluxului beneficiază de protecție.

Din perspectiva costurilor, fluxul ZKP complet are o latență medie de 64,76 milisecunde, comparativ cu sub 2,23 milisecunde pentru fluxurile OAuth. Acest compromis este justificat de eliminarea completă a riscului de exfiltrare a parolei.

## **Slide 14 – Comparație cu protocoalele PAKE: SRP-6a și zk-SNARK**

Deoarece o comparație directă între protocolul ZKP și fluxurile OAuth 2.0 nu reflectă cu fidelitate poziționarea soluției în peisajul protocoalelor fără divulgare a secretului, s-a realizat o evaluare complementară față de SRP-6a, standardizat prin RFC 5054, și zk-SNARK, prin intermediul bibliotecii pysnark.

Pentru echitatea comparației, implementarea Schnorr a fost adaptată pe curba eliptică secp256r1, cu verificare mutuală și derivarea unei chei de sesiune prin ECDH.

Rezultatele pe 100 de iterații indică: pysnark înregistrează 1,92 milisecunde, SRP-6a prezintă 42,27 milisecunde, Schnorr-EC se poziționează la 38,09 milisecunde, iar Schnorr-EC-C-Lib atinge 9,51 milisecunde. Protocolul Schnorr-Pow, operând pe grupul modular original, înregistrează costul cel mai ridicat, de 155,91 milisecunde. Toate protocoalele evaluate asigură netransmiterea parolei prin rețea. SRP-6a și Schnorr asigură autentificare mutuală și derivarea unei chei de sesiune. SRP-6a oferă suplimentar rezistență la compromiterea bazei de date, proprietate pe care varianta Schnorr curentă nu o implementează nativ. Ca nivel de securitate, SRP-6a operează la aproximativ 112 biți, iar variantele Schnorr-EC ating 128 de biți. Protocolul zk-SNARK prezintă latența cea mai redusă datorită naturii neinteractive a demonstrației, dar presupune o fază de configurare inițială de încredere ale cărei costuri nu sunt reflectate în măsurători.

## **Slide 15 – Direcții de dezvoltare**

Ca direcții prioritare de dezvoltare se impun: adoptarea unor algoritmi robuști de derivare a secretului cu factor de cost ajustabil, precum scrypt cu parametri OWASP sau Argon2. De asemenea, se propune migrarea fundamentului matematic de la aritmetica modulară către criptografia pe curbe eliptice, unde schema Schnorr operează nativ cu parametri de dimensiuni reduse. Reimplementarea nucleului criptografic într-un limbaj compilat, precum Rust sau C, ar reduce semnificativ suprasarcina interpretorului Python. Explorarea algoritmilor criptografici post-cuantici ar consolida reziliența sistemului în fața atacurilor de tip Store Now, Decrypt Later. Trecerea de la semnături simetrice HS256 către algoritmi asimetrice RS256 sau ES256 ar decupla procesele de emitere de cele de verificare a jetoanelor.

## **Slide 16 – Concluzii**

Implementarea demonstrează viabilitatea autentificării web fără transmiterea parolei către server. Protocolul satisface rigorile funcționale, de securitate și de performanță, iar integrarea în ecosistemul OAuth confirmă compatibilitatea cu standardele existente. Evaluarea comparativă cu SRP-6a și zk-SNARK poziționează soluția într-o zonă intermediară de performanță, confirmând competitivitatea sa cu protocoalele PAKE consacrate.

Ca limitări, parametrii actuali, precum  $G = 4$  și derivarea simplificată prin SHAKE256 în browser, sunt adecvați exclusiv prototipului. Ca direcții viitoare, se impun tranziția către curbe eliptice, derivarea cheilor prin Argon2, reimplementarea în limbaj compilat, externalizarea sesiunilor către Redis, semnături asimetrice pentru jetoane, implementarea obligatorie a HTTPS și încorporarea jetoanelor de tip proof-of-possession.

## **Slide 17 – Bibliografie selectivă**

Acestea sunt câteva referințe din bibliografia utilizată.

## **Slide 18 – Mulțumesc**

Vă mulțumesc pentru atenție!